Notes

Table of Contents

From Kernel Schedulers to BPF . . . 2
- What is a process Scheduler . . . 2
- Notes . . . 2

Sched-Ext (Arighi Blogpost) . . . 2
- What is Sched-Ext . . . 2
- State in linux . . . 2
- Notes . . . 3

David Vernet (Tech Over Tea interview) . . . 3
- Scheduler's problem space . . . 3
- CFS . . . 3
- EEVDF . . . 3
- Sched_ext . . . 4
- What are problem Sched_ext solve . . . 4
- Notes . . . 5

David Vernet (The Linux Foundation) . . . 5
- Introduction . . . 5
- How to implement new scheduling policies . . . 6
- Examples . . . 6
- Code path . . . 7
- Common objections (less important) . . . 7
- Change to BPF needed for sched ext (less important) . . . 8
- Notes . . . 8

Methods Documentation . . . 8
- Introduction to concept . . . 8
- Scheduling with BPF . . . 8
- Callbacks and functions . . . 9
- Scheduling cycle . . . 10

Extra . . . 11
- Links . . . 11
- To search . . . 11
- To add . . . 11

History . . . 11

Practical Notes . . . 12
- Scheduling class declaration . . . 12
- Partial vs not . . . 12
- Changing policy of a programs . . . 13

Setting time slice (+ mask/cpu) in the scheduler . . . 13
Important components of sched_ext . . . 14
Scx Simple statistics . . . 15
AsciiDoc . . . 16

From Kernel Schedulers to BPF

Summary of the Blog (Link to blog)

What is a process Scheduler

A process scheduler is an essential part of any OS which does:

1. **Task selection (/ranking)**: choose the task to be run next
2. **Time Slice**: choose how long is the next task allowed to run
3. **Core selection**: choose which cpu core should be use

Skipped conflicting, CFS EEVDF ...

Notes

Skipped notes of middle part.

Sched-Ext (Arighi Blogpost)

Implement your own kernel CPU scheduler in Ubuntu with sched-ext

Summary of the Blog (Link To Blog)

What is Sched-Ext

sched-ext is a new extensible scheduler class introduced in the Linux kernel that provides a mechanism to implement **scheduling policies as BPF programs** (Berkeley Packet Filter). This **"pluggable"** approch make possible to easily and quickly implement test scheduling policies (there is no need to reboot the system).

Such programs can also be **connected to user-space** counterparts to defer scheduling decisions to regular user-space processes.

State in linux

sched-ext is **not yet upstream** in the Linux Kernel, it is only available as a patch set in the Linux kernel mailing list (because of concern and potential issue). Is it still possible to use by having a patched kernel with sched-ext but it is still **experimental**.

Notes

Skipped example.

David Vernet (Tech Over Tea interview)

Linux Kernel Scheduler Developer | David Vernet (Tech Over Tea)

Summary of video (Link to Video)

already inside?

Scheduler's problem space

Every machine has a finite number of cores and job of the sheduler is to decide **which process to run on which core**. The scheduler want to balance the workload, running as many process in parallel as possible, but it also has to account for the fact that moving a process to another core away from warm/hot caches, could increase pagefault and reduce overall performance.

It also require a **load balancing**, which devide the load amonst the cores. The load is how much things are running on the system, calculated as weight (normally 100 for a thread) * time that can run for. The objective is to devide the load as equally as possible (while keeping in mind warm cache). Often is done periodically, so for some time load accumulate than it is balance,... After deviding the load over various core, each core has its own run queue and calculate its own thread vrnuntime.

Other problems of the scheduler are **fairness** (per core) and **Interactivity**. A good metrics for the scheduler is IPS meaning Instruction Per Seconds.

The linux kernel initally used **CFS** (Completely Fair Scheduler) from 2006/2007(?) until 6.6 and then it switched to **EEVDF** (Earliest Eligible Virtual Deadline First).

Scheduler are becaming harder to write because of more various hardware and workload. For example, a VM has "virtual core" which are seen to the scheduler as threads, and in certain situation it is better to fix this threads to single core and avoid as much as possible to stop these threads.

already? check like othe

CFS

The idea is to count **Virtual Runtime**, the time the process run for in total, and then you want to give to the one that has run for the least amount of time. In this calculation **priority (niceness)** is used to give more time to process with higher priority by inverve scaling of time accumulated (divide by priority).

EEVDF

Still has the Virtual Runtime but in addition it has a **deadline** which is summed to the Virtual Runtime to obtain a higher level of interactivity.

maybe add it?

Sched_ext

sched_ext and is a new **pluggable scheduling framework** (in the work for the last 2 years), which lets you implement your own host wide scheduling policies as what are called BPF programs.

1. **Host wide**: it migrate all thread to your own scheduler except the real-time(/deadline) threads, which are higher priority and remain in the kernel.
2. **eBPF**: let you run a jit (Just In Time) in the kernel, it has a specific istruction set and encoding for the istruction. During runtime it will read this BPF byte code and emit x86,arm,... istructions to run the program in to the kernel.
   a. There is a backend for the llvm (Compiler backend for multiple programming languages) that can compile c code into BPF instructions.
   b. Initially the name come from Packet Filtering (BPF = Berkeley Packet Filter) but then it extended (eBPF) and now it is no longer an acronym for something.
   c. BPF programs **cannot crash (theoretically) the kernel** because it is statically analysed by the **verifier** and it it could be potentially unsafe (like reading a pointer that shouldn't be read, has a memory leak,...) then it won't load the program into the kernel.
   d. Can **interact with userspace** from your BPF program with data structure called **maps**, real time shared memory readable and writible from both userspace and kernel space.

What are problem Sched_ext solve

CFS and EEDVF are general purpose, fair and worked for many years, but:

1. it it **hard to test**, because of long compilation time and possibility of breakage. BPF program are faster to recompile entire kernel and can dynamically loaded.
2. cannot have a highest performace for **specific usecase** because it need to be general purpose.
3. api it is **difficult to understand** and require low level knowlenge. Meanwhile api in sched_ext are more intuitive and abstract some of the complexity.
4. is it fixed to **stay in the kernel**, in the kernel there is no floating point math (only fixed point), so complex component such as load balancing cannot be offloaded to userspace like is it possible with sched_ext.

- the **overhead** of sched_ext exist, for example for the translation of the BPF istruction but it is **minor** (a couple tens of a percentage)
  - That is because **BPF is not userspace** framework, instead the kernel call your program when still being in kernelspace so the most important part remain in the kernel with no handshake but you can build userspace component above that (often load balancing is done in userspace for ease, and it has minor overhead because it is run only every 1-2s).
  - An exception to it is the work of Andrea Righi, a Linux Kernel Engineer at Canonical, which is writing a linux scheduler written in Rust that **runs in user-space**.

already inside ? controllo 101

Kernel space vs user space

User space is the default, each normal process is runned as such. Processes in user space have their own **virtual address space**.

Kernel space processes are the one that manage the system, for example mapping the virtual memory to physical, scheduling,... The kernel has its own **monolithic address** space every thread in the in the kernel is in the same little sandbox but that sandbox is like the management of the system, it's distributing resources and it's a privileged component.

Passing between kernel space and user space is a **delicate**, because it can be cause of vulnerability and **expensive** operation, because a lot of things are happening in hardware. The execution context is changed, your registers the the user space registers are being saved on the stack; you're changing where the instruction pointer to point to somewhere in the kernel; you have to copy memory from user space; you're changing privilege level so all this stuff um that's called **trapping into the kernel** that's the term for all of this, which happens every time you go back and forth between the two.

Notes

1. They are trying to standardize the BPF woth the help of the **IETF (Internet Engineering Task Force)** especially for hardware vendor which would like to offload BPF program to external device.
2. One vruntime queue for cpu can scale well but doing load balancing (which is expensive) need to iterate over all vruntime queue, so it exist a **shared fifo run queue**, can be used for faster load balancing.

David Vernet (The Linux Foundation)

sched ext | David Vernet (The Linux Foundation)

Summary of video (Link to Video)

Introduction

David Vernet works at meta. He has been working for Meta on sched_ext, for the past year and half (in date june 2023). sched_ext enable scheduling policies to be implemented in BPF programs. Its main advantages are:

1. **Rapid experimentation** needed for new and diverse hardware and software
    a. **Simple** and intutitive API
    b. **Safe**, cannot crash host (afforded by BPF verifier and by watchdog that boots scheduler if it hasn't scheduled some runnable task after some time)
2. Allows creating scheduler **optimized for specific services** (bespoke schedulers)
    a. at meta allowed to optimize FB web service by **1.25-3% RPS, 3-6% P99 latencty**.

3. **Quick rollouts** of new policies (es. temporary workaround before full mitigation)
4. Allows moving policy decision to **user space**

How to implement new scheduling policies

The BPF program must:

1. Implements a set of callbacks
   a. Task wakeup
   b. Task enqueue/dequeue
   c. Task state change (runnable, running, stopping, quiescent)
   d. CPU need task(s) (balance)
   e. Cgroup integration
   f. ...
2. provide fields which globally configure scheduler
   a. **max number of tasks** that can be dispatched
   b. **timeout** in ms (not exceeding 30s)
   c. **name** of scheduler

It is actually not require to implement any of the methods because each of them have a **default implementation**.

Examples

▼ *Examples of methods to implements*

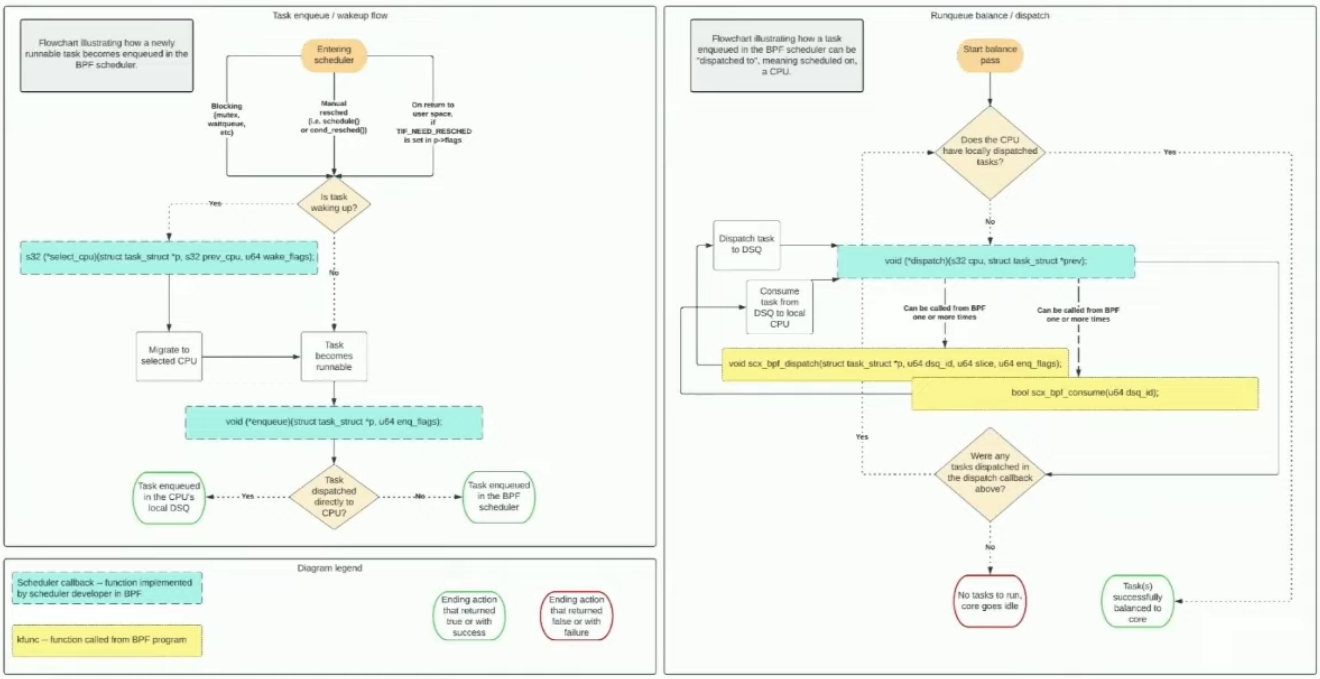
```
/* Return CPU that task should be migrated to on wakeup path. */
s32 (*select_cpu)(struct task_struct *p, s32 prev_cpu, u64 wake_flags);

/* Enqueue runnable task in the BPF scheduler. May dispatch directly to CPU. */
void (*enqueue)(struct task_struct *p, u64 enq_flags);

/* Complement to the above callback. */
void (*dequeue)(struct task_struct *p, u64 deq_flags);
...

/* Maximum time that task may be runnable before being run. Cannot exceed 30s. */
u32 timeout_ms;

/* BPF scheduler's name. Must be a valid name or the program will not load. */
char name[SCX_OPS_NAME_LEN];
```

▼ *Examples of partial implementation*

```
const volatile bool switch_partial; /* Can be set by user space before loading the program. */

s32 BPF_STRUCT_OPS(simple_init)
{
        if (!switch_partial) /* If set, tasks will individually be configured to use the SCHED_EXT class. */
                scx_bpf_switch_all(); /* Switch all CFS tasks to use sched_ext. */
        return 0;
}

void BPF_STRUCT_OPS(simple_enqueue, struct task_struct *p, u64 enq_flags)
{
        if (enq_flags & SCX_ENQ_LOCAL) /* SCX_ENQ_LOCAL could be set if e.g. the current CPU has no other tasks to run. */
                scx_bpf_dispatch(p, SCX_DSQ_LOCAL, enq_flags);  /* Dispatch task to the head of the current CPU's local FIFO. */
        else
                scx_bpf_dispatch(p, SCX_DSQ_GLOBAL, enq_flags); /* Dispatch task to the global FIFO, it will be consumed
                                                                 * automatically by ext. */
}

void BPF_STRUCT_OPS(simple_exit, struct scx_exit_info *ei)
{
        bpf_printk("Exited"); /* Can do more complicated things here like setting flags in user space, etc. */
}

SEC(".struct_ops")
struct sched_ext_ops simple_ops = {
        .enqueue                = (void *)simple_enqueue,
        .init                   = (void *)simple_init,
        .exit                   = (void *)simple_exit,
        .name                   = "simple",
};
```

Code path

See video.



Common objections (less important)

1. will **kill CFS contribuitions**, staying out of tree and non-GPL
    a. verifier check that is GPL
2. **difficult for distro** to support
    a. possibly need to taint the kernel

3. will **force UAPI** onto the scheduler

Change to BPF needed for sched ext (less important)

1. **bpf_cpumask** is a wrapper around the internal cpumask_t (comparable with cpumask_t directly)
2. use RCU context instead of **kptr_get**

Notes

Rewatch part of example and code path (5-10 minutes mark)

Methods Documentation

Data from LWN's article (Link) and sched-ext documentation (Link)

Introduction to concept

The core idea behind BPF is that it allows programs to be loaded into the kernel from user space at run time.

The linux scheduler is not a single one, as there are the **CFS, realtime and deadline scheduler**, even though the majority of the program are managed by the CFS. Sched-ext allow to have dedicate scheduler for workload.

Scheduling with BPF

The proposed patch adds a **new scheduling class** called SCHED_EXT that can be selected with `sched_setscheduler()` call like others (except SCHED_DEADLINE).

It is an **unprivilaged** class so any process can put themself in this class and it is placed **between SCHED_IDLE and SCHED_NORMAL**. This make it impossible to take over the system with a SCHED_EXT scheduler, preventing application from running as SCHED_NORMAL. Also, it suggest that when a SCHED_EXT scheduler is load most the process will be handled by this, if no such scheduler is loaded process in the SCHED_EXT are run with as if they where is SCHED_NORMAL. A BPF scheduler can **move all running process into SCHED_EXT** (only the ones below realtime priority) with the call `scx_bpf_switch_all()`.

Dispatch queues

A BPF scheduler will manage a set of **dispatch queue** (DSQ), which contains runnable tasks waiting for a CPU, they are a way to matchthe impedance between the scheduler core and the BPF scheduler. They can operetare as FIFO or priority queue.

By default, there is **one global** FIFO (`SCX_DSQ_GLOBAL`), and **one local per CPU** (`SCX_DSQ_LOCAL`), but it can manage an arbitrary number of DSQs using `scx_bpf_create_dsq()` and `scx_bpf_destroy_dsq()`.

These are used as such:

1. CPU always **executes a task from its local DSQ** (the first one). A task is "dispatched" to a DSQ. A non-local DSQ is "consumed" to transfer a task to the consuming CPU's local DSQ.
2. if during this search the local DSQ is empty, the CPU **tries to consume the global DSQ**. If that doesn't yield a runnable task either, `ops.dispatch()` is invoked.

Callbacks and functions

A BPF scheduler is mostly implemented as a **set of callback to be invoked via an operations structures**, every of which inform the BPF of an event of a decision to be made. A BPF scheduler is a set of BPF programs that implements the **strct sched_ext_ops**. The only actual required field is `ops.name` which must be a valid BPF object name, all the rest have default implementations. Some of the callback are:

1. `prep_enable(), enable()`: inform that a process is **entering SCHED_EXT** class, and can be used to set up associated data. The first call is allowed to block and is used for memory alocations, while the second cannot block but is the one to actually enabling scheduling for the new task.
2. `select_cpu()`: **select a CPU for a task that is just waking up**, it should return the number of the CPU to place the task on (which can be changed again before running it). This decision may be used by the scheduler to wake the selected CPU if it is currently idle.
3. `enqueue()`: **Enqueue a task into the scheduler** for running. Normally it call `scx_bpf_dispatch()` to place a task into a chosen dispatch queue, from which it will eventually be run.
   - this call should provide the **lenght of time slice** the task will have once it runs.
     - If the speciefied as SCX_SLICE_INF, it will go into **tickless mode**
   - enqueue is **not required** to put in any dispatch queue and it could stash it somewhere for sometime. But the kernel ensure that a process is not forgotten for more than 30 seconds (by default, can be reduced and not increased).
4. `dispatch()`: called on **empty CPU dispatch queue** to assign new task to keep cpu busy.
   - If it remain empty the scheduler (?) will try to grab some from the global queue.
5. `update_idle()`: inform the scheduler of a **CPU leaving or entering idle**.
6. `runnable(), running(), stopping(), queiscent()`: inform the scheduler of a **status changes for a task** (call respectively when a task: become runnable, start running on CPU, is taken off a CPU, is no longer runnable).
7. `cpu_acquire(), cpu_release()`: inform scheduler of **availability change of a CPU**. Example of cpu_release is when a realtime scheduling class claim a CPU to manage.

- Full list at github link sched/ext.h, plus other resourse at end of sched-ext rust documentation.

Functions

There are also **function** that can be called by the scheduler can call to affect scheduling decision. For example `scx_bpf_kick_cpu` can be called to preempt a task tha is running on a CPU (removing it) and them call back the scheduler to pick a new task to run there.

Scheduling cycle

Scheduling Cycle

The following briefly shows how a **waking task is scheduled and executed**.

1. Firsty `ops.select_cpu()` is invoked for two reason
   a. The CPU selected by ops.select_cpu() is an **optimization hint and not binding**. The actual decision is made at the last step of scheduling. However, if it match where it runs there is a small performance gain.
   b. A side-effect is **waking the CPU from idle**. it up from idle.
      - While a BPF scheduler can wake up any cpu using call `scx_bpf_kick_cpu()`, using ops.select_cpu() judiciously can be **simpler and more efficient**.
   c. A task **can be immediately dispatched** to a DSQ by calling `scx_bpf_dispatch()`.
      - If it is dispatched to `SCX_DSQ_LOCAL`, it will be dispatched to the local DSQ of whichever **CPU is returned** from `ops.select_cpu()`.
      - Additionally, dispatching directly will cause the `ops.enqueue()` **callback to be skipped**.
   d. Note that the scheduler core will **ignore an invalid CPU** selection, for example, if it's outside the allowed cpumask of the task.
2. Then `ops.enqueue()` is invoked which make **one of the following decisions**:
   a. **Dispatch to the global or local or custom** DSQ by calling `scx_bpf_dispatch()` with SCX_DSQ_GLOBAL, SCX_DSQ_LOCAL or a DSQ ID smaller then 2^63, respectively.
   b. **Queue the task** on the BPF side.
3. When a CPU is ready, search firstly the local then the global DSQ. **If there still isn't a task to run**, `ops.dispatch()` which can use:
   a. `scx_bpf_dispatch()` **dispatches a task to a DSQ**. Any target DSQ can be used - SCX_DSQ_LOCAL, SCX_DSQ_LOCAL_ON | cpu, SCX_DSQ_GLOBAL or a custom DSQ.
      - Currently can't be called with BPF locks held, this is being worked on and will be supported.
      - **schedules dispatching** rather than performing them immediately.
      - There can be **up to ops.dispatch_max_batch pending** tasks.
   b. `scx_bpf_consume()` tranfers a task from the specified non-local DSQ **to the dispatching DSQ**.
      - This function cannot be called with any BPF locks held.
      - **flushes the pending dispatched tasks** before trying to consume the specified DSQ.
   c. If are used **only the built-in DSQs there is no need to implement** this callback as a task is never queued on the BPF scheduler and both the local and global DSQs are consumed automatically.
4. After ops.dispatch() returns,
   a. search for **tasks in the local then global DSQ**

b. If `ops.dispatch()` **has dispatched any tasks, retry #3**.

c. If the **previous task is a runnable SCX task**, keep executing it (see SCX_OPS_ENQ_LAST).

d. Go **idle**.

Extra Notes

1. `scx_bpf_dispatch()`/`scx_bpf_dispatch_vtime()` queues the task on the **respectevly on FIFO/priority**, queue of the target DSQ.
2. Internal DSQs such as **SCX_DSQ_LOCAL and SCX_DSQ_GLOBAL do not support priority-queue**.

Extra

Links

1. sched-ext/scx
2. extensible scheduler class (LWN)
3. Schedulers, pluggable and realtime (LWN)
4. user-space scheduler (Arighi Blog)

To search

1. Scheduling class
2. Operations structures (Link)
3. Tickless mode → no timeout i think (Link)
4. BPF maps
5. What does every scheduler example do
6. Is BPF rbtree still out of tree (starting point is link)

To add

1. Selecting mode partial with -p in examples

History

1. finding information about how it works in broad strokes
2. trying to set the scheduling class to a process with setattr/getattr
3. filing bug for sccache (Link to issuetracker)

Practical Notes

Scheduling class declaration

It is declared useing operational structure (?) with link to the callback used plus **name**, **flags** and **timeout_ms**

▼ *Details*

```
SEC(".struct_ops.link")
struct sched_ext_ops rustland = {
    .select_cpu     = (void *)rustland_select_cpu,
    .enqueue        = (void *)rustland_enqueue,
    .dispatch       = (void *)rustland_dispatch,
    .running        = (void *)rustland_running,
    .stopping       = (void *)rustland_stopping,
    .update_idle        = (void *)rustland_update_idle,
    .set_cpumask        = (void *)rustland_set_cpumask,
    .cpu_release        = (void *)rustland_cpu_release,
    .init_task      = (void *)rustland_init_task,
    .exit_task      = (void *)rustland_exit_task,
    .init           = (void *)rustland_init,
    .exit           = (void *)rustland_exit,
    .flags          = SCX_OPS_ENQ_LAST | SCX_OPS_KEEP_BUILTIN_IDLE,
    .timeout_ms     = 5000,
    .name           = "rustland"
};
```

Partial vs not

Sched_ext scheduler in bpf can choose what they manage thanks to the **flag in enum scx_ops_flags** (Code link). The flag we care is SCX_OPS_SWITCH_PARTIAL which is set to `1LLU << 3` (may change in the future), this is probably used as a mask to get a single bit representing the if the option is set or not.

This flag can be changed in the bpf programs with the command `skel→rodata→switch_partial = true;` that does the following:

1. If set, **only tasks with policy set to SCHED_EXT** are attached to sched_ext.
2. If clear, **SCHED_NORMAL tasks are also included**.

This is easily changable in the example scheduler thanks to the **-p** option. This option though is being removed, possibly because it is **not recommended to run in partial** mode on a production system.

Extra info that could be usefull

1. Look at void print_scx_info(const char *log_lvl, struct task_struct *p); at link
2. Why is the option -p being removed.

Changing policy of a programs

For changing policy of a program it can be used the `get[/set]attr` command, or simply get[/set]sched could be used. They also have respective c binding that are named as `sched_*`.

That mean it is also possible to do the same on rust, which I did. My program also allow to see a list of program on different scheduler filtering the output of the `ps -Ao` command. It is also possible then to revert all program to SHED_NORMAL.

A list of all the different scheduling class can be seen here:

```
SCHED_NORMAL = 0
SCHED_FIFO = 1
SCHED_RR = 2
SCHED_BATCH = 3
/* "SCHED_ISO": reserved but not implemented yet */
SCHED_IDLE = 5
SCHED_DEADLINE = 6
SCHED_EXT = 7
```

Setting time slice (+ mask/cpu) in the scheduler

In bpf, we can set a time slice **directly while dispatching** the task. This is done with the method `scx_bpf_dispatch(task, dispatch_queue__id, time_slice_ns, enque_flags)`.

1. By default is often SCX_SLICE_DFL = 20 * NSEC_PER_MSEC
2. Can be set to imply **nohz** SCX_SLICE_INF = U64_MAX

- This data can be found at shed/ext.h

In rustland_core which is a base for some (maybe all) examples scheduler there is a more complex method that invoke the first one, called `dispatch_task(struct task_struct *p, u64 dsq_id, u64 cpumask_cnt, u64 task_slice_ns, u64 enq_flags)` which also have a cpumask.

In this last method, if the dsq_id is neither SCX_DSQ_LOCAL nor SHARED_DSQ than dispatch a task to a **specific per-CPU** DSQ if the target CPU can be used (according to the cpumask), otherwise redirect the task to the first CPU available, using the shared DSQ logic.

Notes

1. In dispatch_task (Code link) it use something like `x ?: y` this is a GNU C extension that is equivalent to `x ? x : y`

2. the config used to enable or disable sched_ext is `CONFIG_SCHED_CLASS_EXT`, also extra config can be found at .github/workflows/sched-ext.config, /* infinite, implies nohz */
3. rust-analyzer constantly autobuild
4. cannot divide in sched_ext

Important components of sched_ext

After a deeper dive, it seems to me that sched_ext scheduler are composed of different component (and/or abstraction layer). They are, from furthest to closest:

1. the **modyfied kernel**, which patch the upstream kernel adding a new scheduling class, sched_ext.
2. **BPF**, **BPF verifier** and i guess (?) **BPF excecuter**
3. **SCX framework** which is mainly composed of scx_rustland_core, which is a basic implementation on which examples are base on and scx_utils, which as the name implies contains utils. This can be found at scx/rust.
4. the **actual scheduler** which is implemented in a *.bpf.c file or bpf.rs file which than is convert to a main.bpf.c file.
    a. Use a **operational structures** (?) which defines callbacks as pointer to functions and other attributes such as name, timeout and flags (see example above).
5. the **actual program** which is responsible for loading and removing of the scheduler from the kernel, making use of the primitive in the SCX_framework. It is also resposible (if required) to log information, warning, error,... about the scheduler.

Rustland Builder

The `build.rs`, which when placed in the root of a package will cause Cargo to compile that script and **execute it just before building** the package.

```
fn main() {
    scx_rustland_core::RustLandBuilder::new()
        .unwrap()
        .build()
        .unwrap();
}
```

Which in turn execute the following code where:

1. **expect**: is like unwrap but panic with the specific error message
2. **include_bytes**: is a macro which will yield an expression of type &'static [u8; N] which is the contents of the file.
3. **enable_intf[/skel]**: (?)
4. **CARGO_MANIFEST_DIR**: The directory containing the manifest of your package, in this case I think it is the manifest of scx_rustland_core, so it is `/rust/scx_rustland_core`

5. **BpfBuilder**: is defined in scx_utils

```
pub struct RustLandBuilder {
    inner_builder: BpfBuilder,
}

impl RustLandBuilder {
    pub fn new() -> Result<Self> {
        Ok(Self {
            inner_builder: BpfBuilder::new()?,
        })
    }

    fn create_file(&self, file_name: &str, content: &[u8]) {
        let path = Path::new(file_name);
        let mut file = File::create(&path).expect("Unable to create file");
        file.write_all(content).expect("Unable to write to file");
    }

    pub fn build(&mut self) -> Result<()> {
        // Embed the BPF source files.
        let intf = include_bytes!(concat!(env!("CARGO_MANIFEST_DIR"),
"/assets/bpf/intf.h"));
        let skel = include_bytes!(concat!(env!("CARGO_MANIFEST_DIR"),
"/assets/bpf/main.bpf.c"));
        let bpf = include_bytes!(concat!(env!("CARGO_MANIFEST_DIR"),
"/assets/bpf.rs"));

        // Generate BPF backend code (C).
        self.create_file("intf.h", intf);
        self.create_file("main.bpf.c", skel);

        self.inner_builder.enable_intf("intf.h", "bpf_intf.rs");
        self.inner_builder.enable_skel("main.bpf.c", "bpf");

        // Generate user-space BPF connector code (Rust).
        self.create_file("src/bpf.rs", bpf);

        // Build the scheduler.
        self.inner_builder.build()
    }
}
```

Scx Simple statistics

The global one seem to count dispatched to global queue, while the local one seem to count the time a task was dispatched to local queue:

```
static void stat_inc(u32 idx)
{
    u64 *cnt_p = bpf_map_lookup_elem(&stats, &idx);
    if (cnt_p)
        (*cnt_p)++;
}

s32 BPF_STRUCT_OPS(simple_mine_select_cpu, struct task_struct *p, s32 prev_cpu, u64
wake_flags)
{
    /* ... */
    if (is_idle) { /* if cpu is idle */
        stat_inc(0);    /* count local queueing */
        scx_bpf_dispatch(p, SCX_DSQ_LOCAL, SCX_SLICE_DFL, 0);
    }
    /* ... */
}

void BPF_STRUCT_OPS(simple_mine_enqueue, struct task_struct *p, u64 enq_flags)
{
    stat_inc(1);    /* count global queueing -> data printed */
    /* ... */
}
```

AsciiDoc

AsciiDoc Quick Reference